

Wk0500 -- ET-ESP32-RS485 V2 Board + Arduino Cloud Platform

Std01.รหัส _____ ชื่อ-สกุล _____

Std01.รหัส _____ ชื่อ-สกุล _____

1. แนะนำ ESP32 Board >>

- https://www.etteam.com/product/I2C_RS485/ET-ESP32-RS485_V2/index.html
- https://www.etteam.com/product/I2C_RS485/ET-ESP32-RS485_V2/man-th-et-esp32-wrover-rs485-v2.pdf

2. ทดสอบการอ่านค่า Modbus RTU Sensor: XY-MD02

3. ทดสอบการอ่านค่า Modbus RTU Sensor: SHT20

- <https://www.cybertice.com/product/4802/sht20-เซ็นเซอร์วัดอุณหภูมิและความชื้น-temperature-and-humidity-transmitter-sht20-rs485-sensor-module>
- <https://www.artronshop.co.th/product/687/เซ็นเซอร์วัดอุณหภูมิและความชื้น-modbus-rs485-sht20>

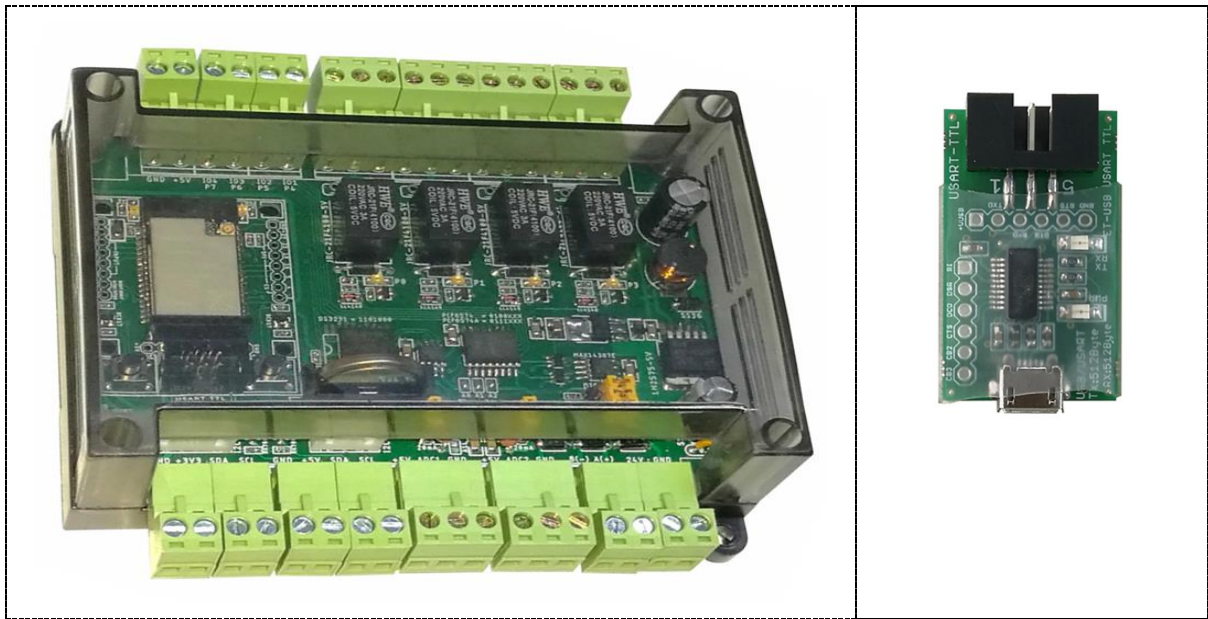
4. ทดสอบการอ่านค่า Modbus Power Meter: SD120

- https://iot-kmutnb.github.io/blogs/sensors/power_meter_sdm120_modbus/



Mission 10.1	Mission 10.2	Mission 10.3	Mission 10.4	Mission 10.5	Mission 10.6	Mission 10.7
Smart Phone + XY-MD02	Smart Phone + SDM120	Smart Phone + White Lamp	Smart Phone + Rly1, Rly2	Smart Phone + Green Lamp + Green Switch	Smart Phone + Yellow Lamp + Black Switch	Smart Phone + Red Lamp + Red Switch

5. ทดสอบการทำงานร่วมกับ Arduino IDE ด้วยโปรแกรม Blink



6. แนะนำ Arduino IoT Platform: <https://cloud.arduino.cc/> และกำหนดให้การทำงานผ่าน Smart Phone ในการทำปฏิบัติการ ให้ใช้ Arduino IoT Platform



7. แนะนำการใช้งาน 24V Pilot Lamp
8. แนะนำการใช้งานสวิตช์แบบกดติดปล่อยดับ แบบ NO{สีดำ, สีเขียว} และแบบ NC{สีแดง}
9. ทดสอบต่อ Lamp และ สวิตช์กับ ESP32



10. ต่อบางจอร์และโปรแกรมระบบให้ทำงาน ดังนี้

- 10.1 อ่านค่าอุณหภูมิและความชื้นแสดงที่ Smart Phone
- 10.2 เพิ่มงานให้ อ่านค่ากำลังไฟฟ้า แรงดัน และกระแส แสดงที่ Smart Phone
- 10.3 เพิ่มงานให้ ควบคุมการปิดเปิด White Lamp ผ่าน Smart Phone
- 10.4 เพิ่มงานให้ ควบคุมการปิดเปิด Relay_1 และ Relay_2 ผ่าน Smart Phone
- 10.5 เพิ่มงานให้ ควบคุมการปิดเปิด Green Lamp ผ่าน Smart Phone และ Green Switch โดย
 - ปิดด้วย Smart Phone
 - เปิดด้วย Smart Phone
 - หาก Lamp On กดสวิตช์จะ Off
 - หาก Lamp Off กดสวิตช์จะ On
- 10.6 เพิ่มงานให้ ควบคุมการปิดเปิด Yellow Lamp ผ่าน Smart Phone และ Black Switch
- 10.7 เพิ่มงานให้ ควบคุมการปิดเปิด Red Lamp ผ่าน Smart Phone และ Red Switch
- 10.8 การควบคุมจาก Smart Phone ให้ใช้ Arduino IoT Platform

Appendix 0/3: Part List

Hardware

1. Plastic Box + Power Adapter + Circuit Breaker
2. ETT-ESP32-RS485 V2.0 Board + ETT USB Loader
3. อลูมิเนียมฉาก + Pilot Lamp + PB(Push Button) Switch
 - Pilot Lamp {RED, GRN, YLW, BLU}
 - PB Switch {BLK, RED, GRN}
4. XY-MD02: Modbus RTU Temperature and Humidity Sensor
5. SHT20: Modbus RS485 Temperature and Humidity Sensor
6. SDM120: MODBUS Power Meter
7. USB-to-RS485 Device
8. RS232-TTL to RS485
9. Relay Socket + Relay24V (x2)
- ~~10. DOIT-ESP32~~
11. สายไฟ

Software

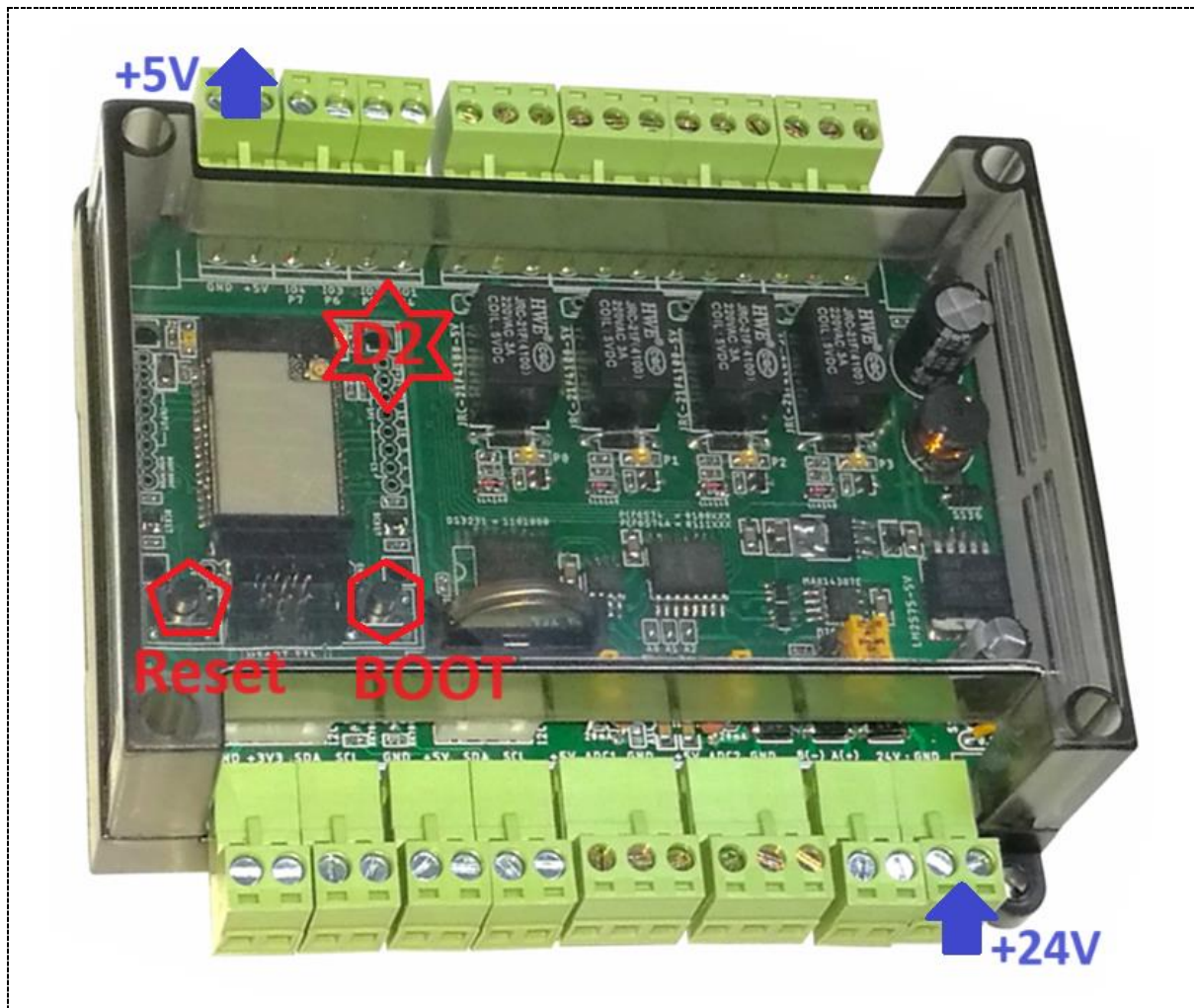
1. Arduino IDE
2. Node-RED
3. Modbus Poll
4. CAS Modbus Scanner

Appendix 1/3: ESP32 Board

ETT-ESP32-RS485 V2.0 Board

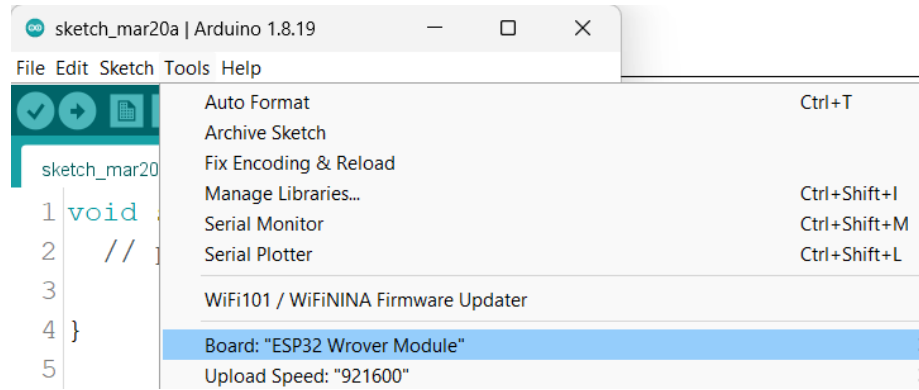
1. แนะนำ ESP32 Board >>

- https://www.etteam.com/product/2C_RS485/ET-ESP32-RS485_V2/index.html
- https://www.etteam.com/product/2C_RS485/ET-ESP32-RS485_V2/man-th-et-esp32-wrover-rs485-v2.pdf



2. Test1 -- Blink

- Select Board = ESP32 Wrover Module



```
#define OnBoardLED 2  
void setup() {  
  pinMode(OnBoardLED, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(OnBoardLED, HIGH); delay(200);  
  digitalWrite(OnBoardLED, LOW); delay(200);  
  digitalWrite(OnBoardLED, HIGH); delay(200);  
  digitalWrite(OnBoardLED, LOW); delay(800);  
}
```

3. Test2 – Relay and GPIO

บอร์ด ET-ESP32(WROVER) RS485 V2 มี Relay แบบหน้าสัมผัส NO/COMMON/NC บรรจุไว้ภายในบอร์ด จำนวน 4ชุด และมี GPIO Pin แบบ 5V Logic อีก 4ชุด โดยเชื่อมต่อผ่าน I2C I/O เบอร์ PCF8574/A

- PCF8574/A ต่อกับ ESP-32 ขา SDA=21, ขา SCL=22 และกำหนด I2C Address = 0x20
- P0, P1, P2, P3 ของ PCF8574/A ควบคุม การทำงานของ Relay P0, P1, P2, P3 (ตามลำดับ)
- P4, P5, P6, P7 เชื่อมต่อกับ GPIO P4, P5, P6, P7 (ตามลำดับ) สามารถกำหนดเป็น Input/Output ได้อิสระจากโปรแกรม

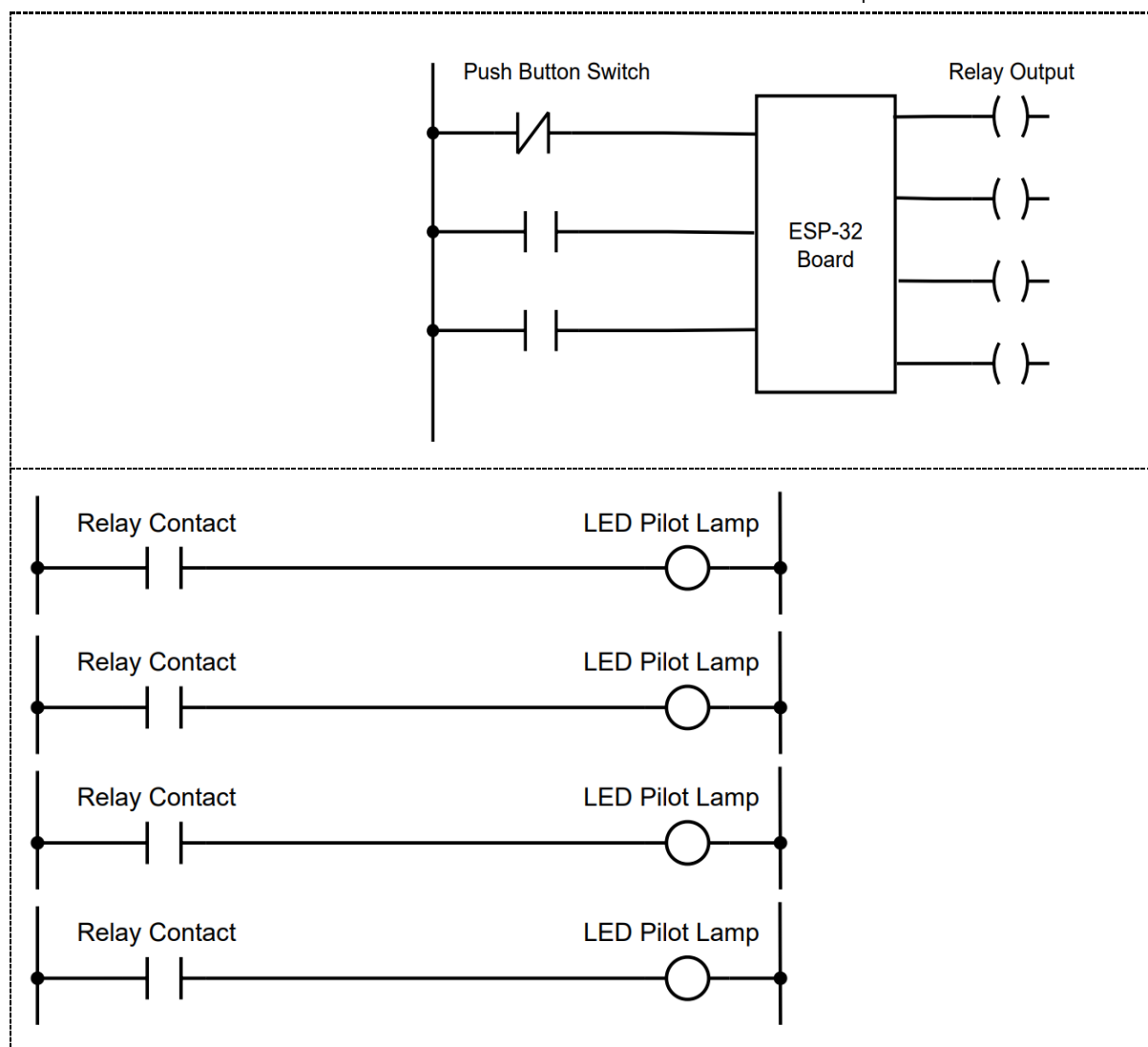
3.1 Read <https://mischianti.org/pcf8574-i2c-digital-i-o-expander-fast-easy-usage/>

3.2 Download library >> https://github.com/xreef/PCF8574_library/archive/master.zip

3.3 Add Library: Sketch → Inc Library → Add Zip...

3.4 ทดสอบการควบคุม Relay และการต่อไปยัง Pilot Lamp

3.5 ทดสอบการอ่านค่า Push Button Switch เพื่อสั่งงาน Pilot Lamp



3.6 Test21 - Relay Control

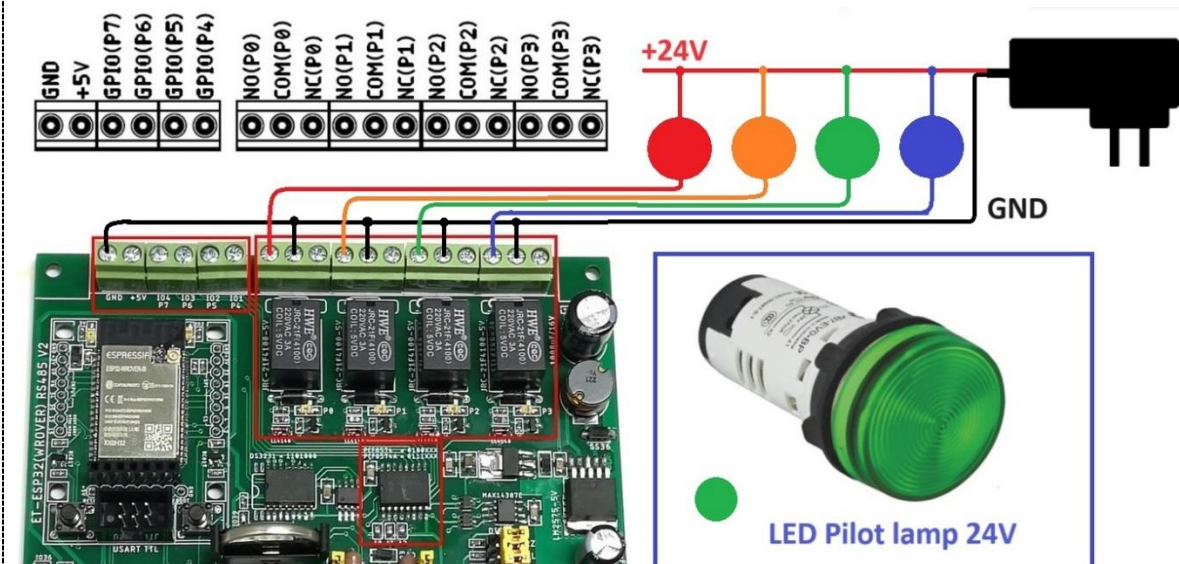
```
#include "Arduino.h"
#include "PCF8574.h" // https://github.com/xreef/PCF8574_library
#define I2C_Address 0x20
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

// Instantiate Wire for generic use at 100kHz
TwoWire I2Ctwo = TwoWire(1);

// Set i2c address
PCF8574 pcf8574(&I2Ctwo, I2C_Address, I2C_SDA_Pin, I2C_SCL_Pin);

void setup() {
  Serial.begin(115200);
  pcf8574.pinMode(0, OUTPUT);
  pcf8574.pinMode(1, OUTPUT);
  pcf8574.pinMode(2, OUTPUT);
  pcf8574.pinMode(3, OUTPUT);
  pcf8574.pinMode(4, INPUT_PULLUP);
  pcf8574.pinMode(5, INPUT_PULLUP);
  pcf8574.pinMode(6, INPUT_PULLUP);
  pcf8574.pinMode(7, INPUT_PULLUP);
  pcf8574.begin();
}

int pin = 0;
void loop() {
  Serial.println(pin);
  pcf8574.digitalWrite(pin, LOW); delay(500);
  pcf8574.digitalWrite(pin, HIGH); delay(500);
  pin = pin >= 3 ? 0 : pin += 1;
}
```



3.7 Test22 - Input

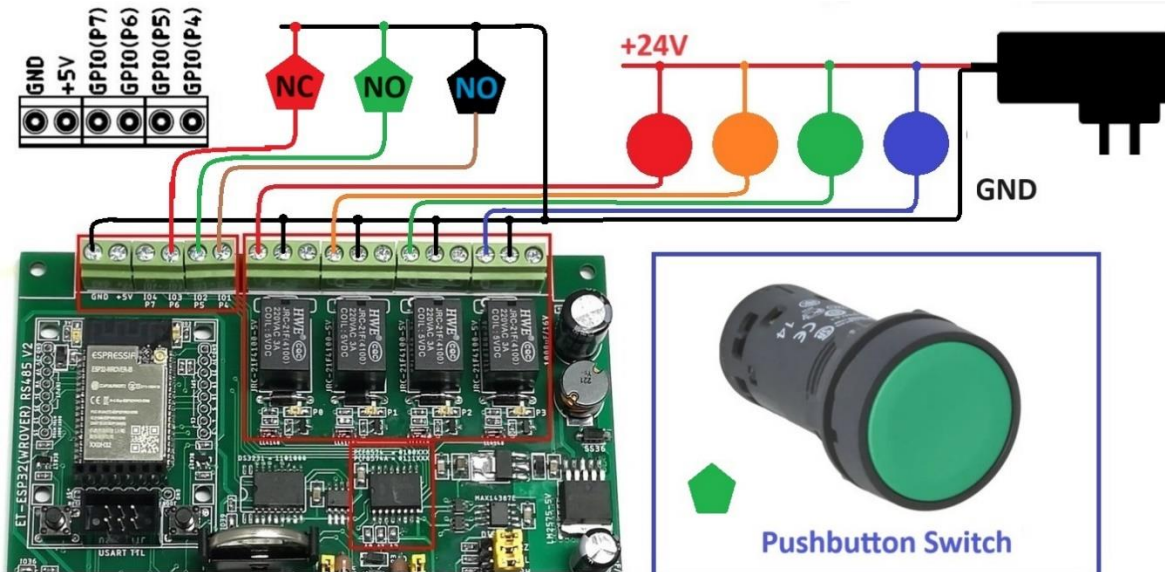
```
#include "Arduino.h"
#include "PCF8574.h" // https://github.com/xreef/PCF8574_library
#define I2C_Address 0x20
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

// Instantiate Wire for generic use at 100kHz
TwoWire I2Ctwo = TwoWire(1);

// Set i2c address
PCF8574 pcf8574(&I2Ctwo, I2C_Address, I2C_SDA_Pin, I2C_SCL_Pin);

void setup() {
  Serial.begin(115200);
  pcf8574.pinMode(0, OUTPUT);
  pcf8574.pinMode(1, OUTPUT);
  pcf8574.pinMode(2, OUTPUT);
  pcf8574.pinMode(3, OUTPUT);
  pcf8574.pinMode(4, INPUT_PULLUP);
  pcf8574.pinMode(5, INPUT_PULLUP);
  pcf8574.pinMode(6, INPUT_PULLUP);
  pcf8574.pinMode(7, INPUT_PULLUP);
  pcf8574.begin();
}

int pin = 0;
void loop() {
  uint8_t val = pcf8574.digitalRead(P4);
  if (val == LOW) Serial.println("KEY PRESSED");
  delay(50);
}
```



3.8 Test23 - Input to Relay

```
#include "Arduino.h"
#include "PCF8574.h" // https://github.com/xreef/PCF8574_library
#define I2C_Address 0x20
#define I2C_SDA_Pin 21
#define I2C_SCL_Pin 22

// Instantiate Wire for generic use at 100kHz
TwoWire I2Ctwo = TwoWire(1);

// Set i2c address
PCF8574 pcf8574(&I2Ctwo, I2C_Address, I2C_SDA_Pin, I2C_SCL_Pin);

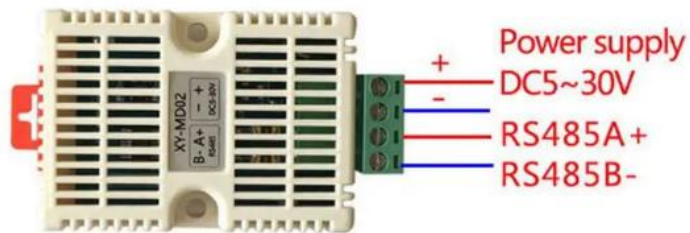
void setup() {
  Serial.begin(115200);
  pcf8574.pinMode(0, OUTPUT);
  pcf8574.pinMode(1, OUTPUT);
  pcf8574.pinMode(2, OUTPUT);
  pcf8574.pinMode(3, OUTPUT);
  pcf8574.pinMode(4, INPUT_PULLUP);
  pcf8574.pinMode(5, INPUT_PULLUP);
  pcf8574.pinMode(6, INPUT_PULLUP);
  pcf8574.pinMode(7, INPUT_PULLUP);
  pcf8574.begin();
}

int Counter = 0;
void loop() {
  if (pcf8574.digitalRead(P4) == LOW)
  { delay(100);
    while (pcf8574.digitalRead(P4) == LOW)
      delay(50);
    Counter++;
    delay(100);
    Serial.println(Counter);
    pcf8574.digitalWrite(P0, Counter % 2);
  }
}
```

Appendix 2a/3: Modbus RTU Read

1/6. Read XY-MD02 from PC

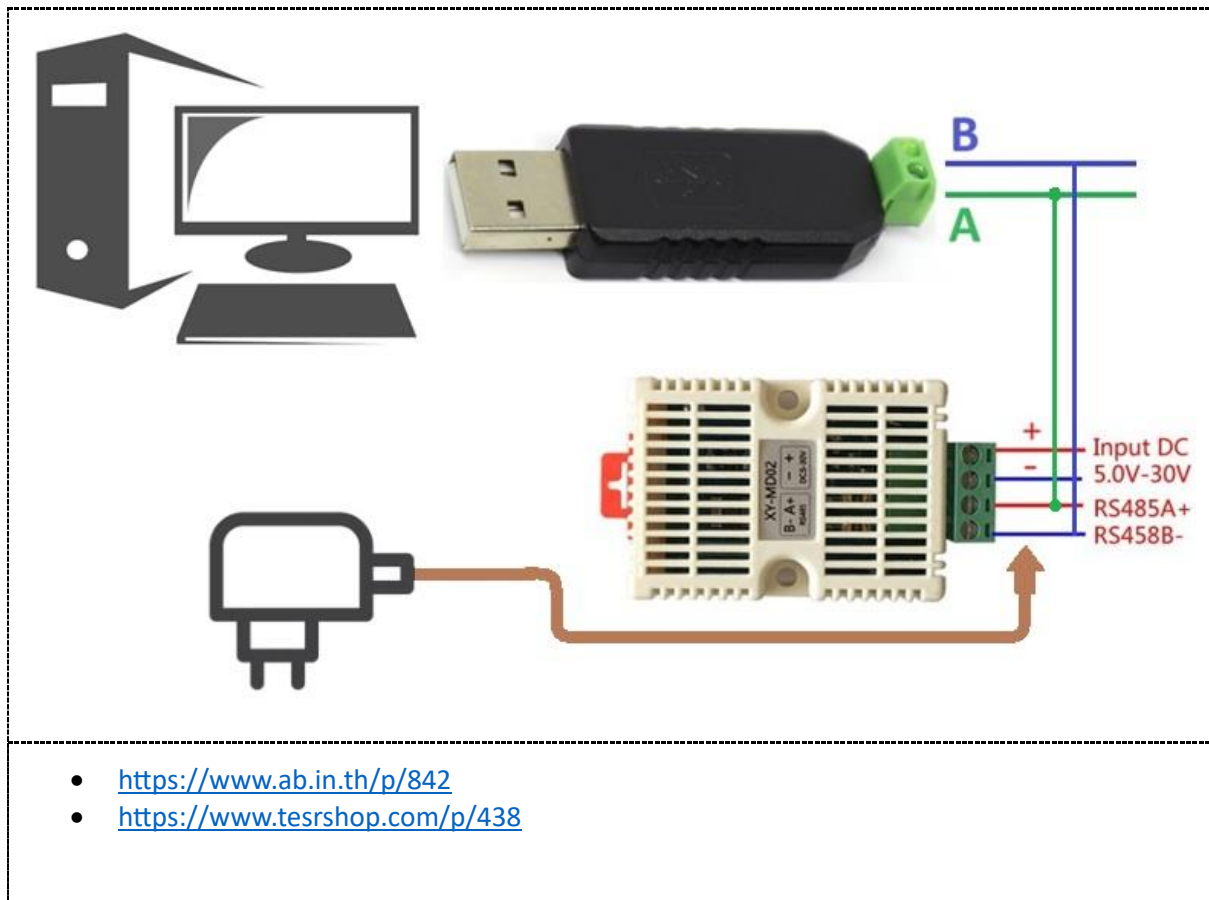
1. แนะนำ DHT22 : <https://www.cybertice.com/p/4530>
2. ทดสอบการอ่านค่า DHT22



RS485 communication distance up to 1000 meters.

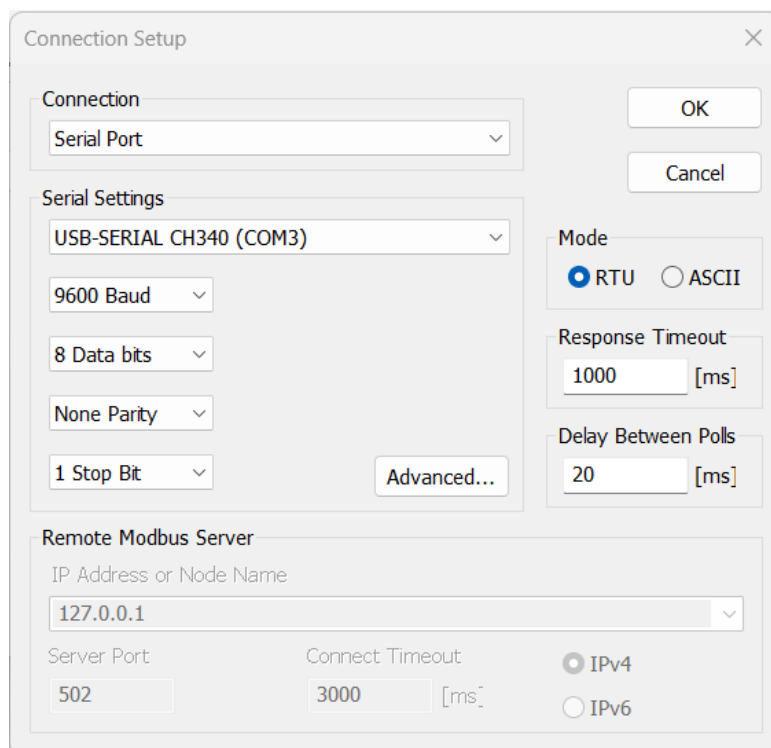
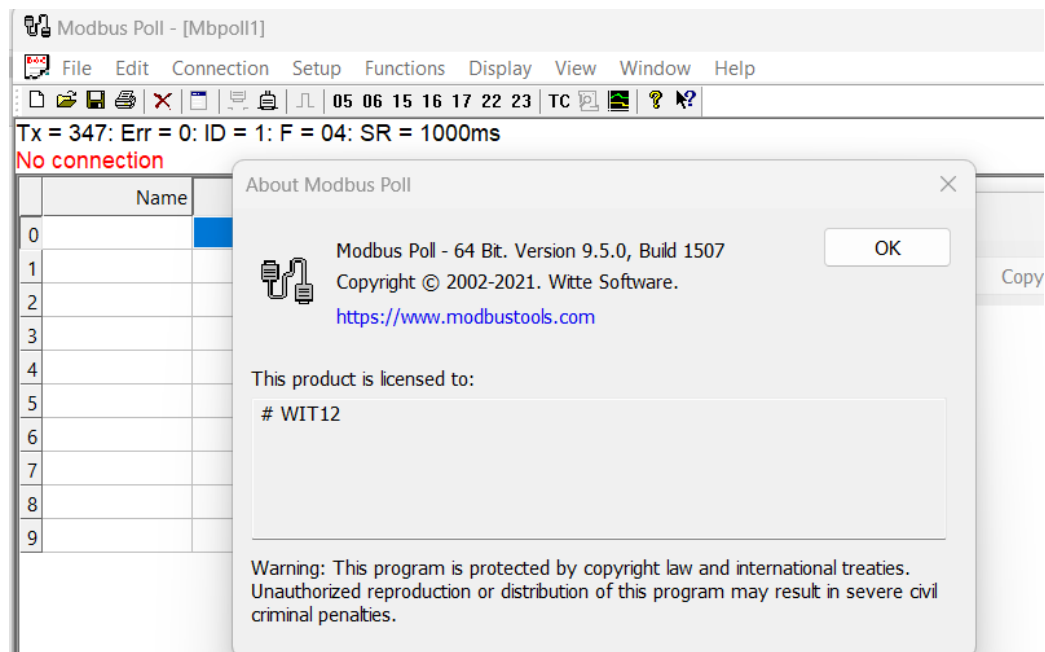
Register Type	Register Address	Register contents	Number of bytes
Input Register	0x0001	Temperature	2
	0x0002	Humidity	2
Keep Register	0x0101	Device Address	2
	0x0102	Baud Rate 0:9600 1:14400 2:19200	2
	0x0103	Temperature correction(/10) -10.0~10.0	2
	0x0104	Humidity correction(/10) -10.0~10.0	2

1. Wiring to PC

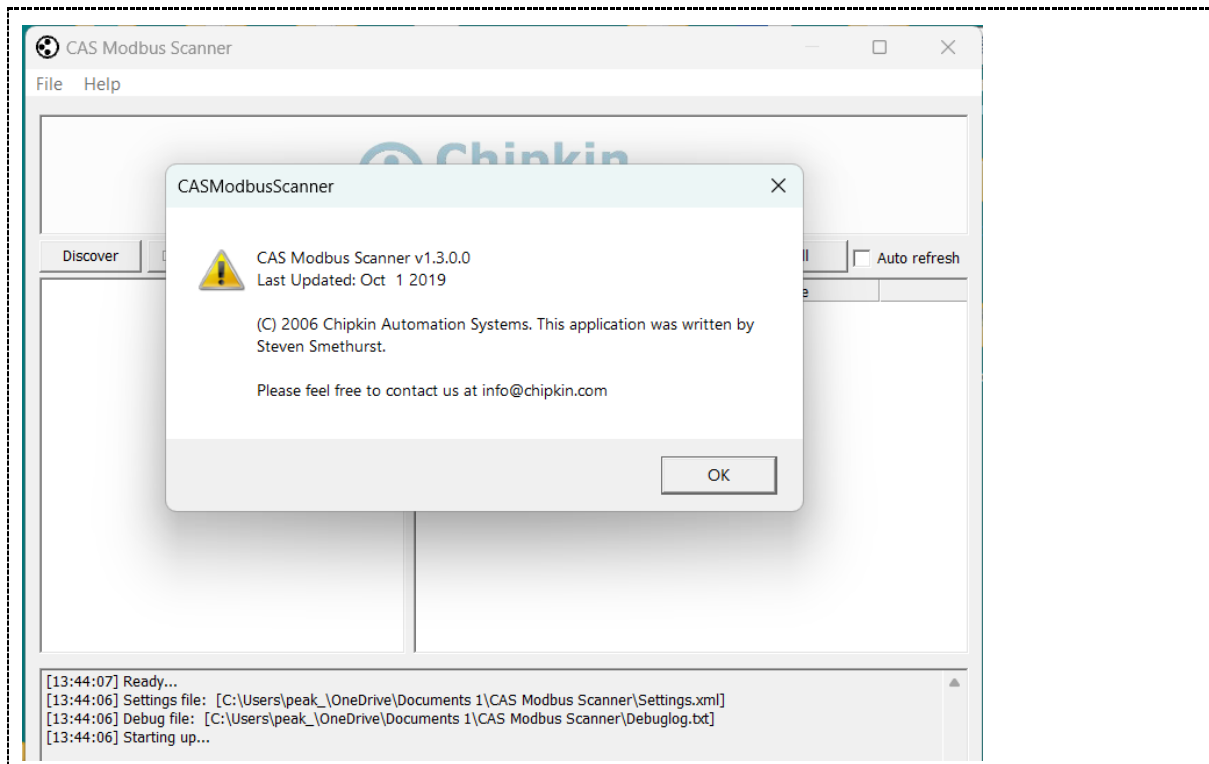


2. Communication Setting

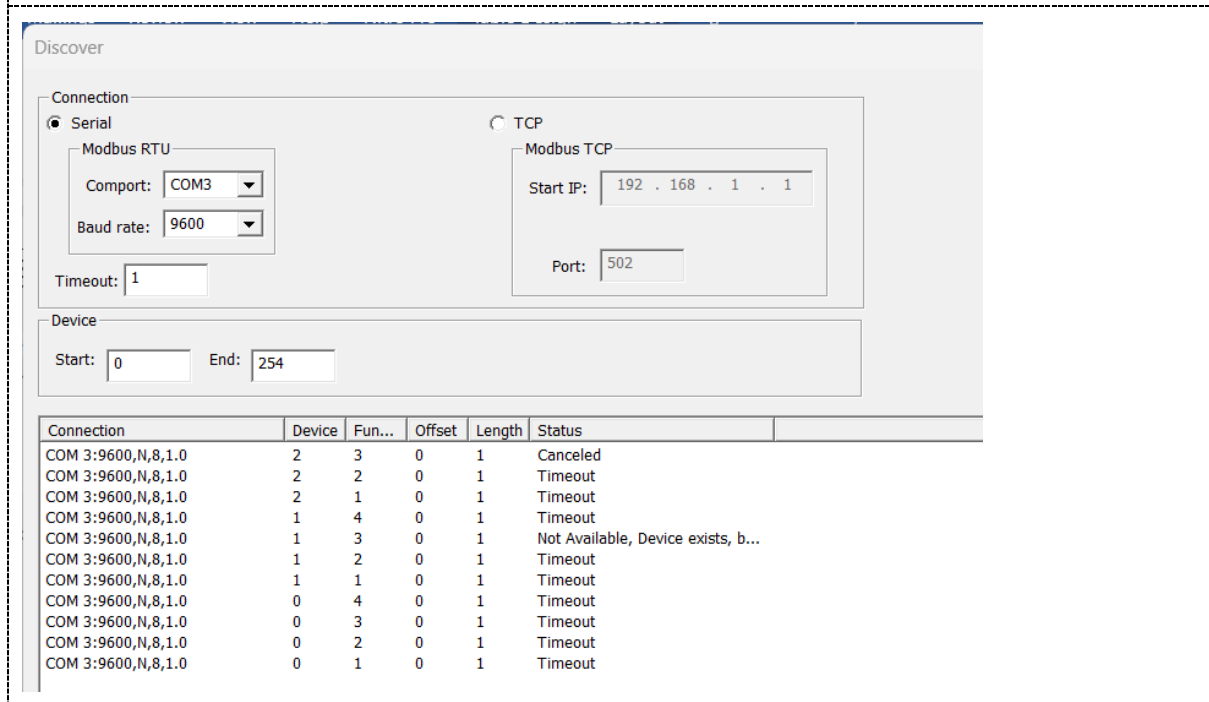
Modbus Poll Software



3. Scan ID by CAS Modbus Scanner v1.3.0.0



Found >> ID = 01 from function 03



4. Read

Slave ID=1, Function = 04, Start = 1, Quantity = 2

Read/Write Definition

Slave ID: OK

Function: Cancel

Address mode
☒ Dec ☐ Hex

Address: PLC address = 30002

Quantity:

Scan Rate: [ms] Apply

Disable
☐ Read/Write Disabled
☐ Disable on error

Read/Write Once

View
 Rows
☒ 10 ☐ 20 ☐ 50 ☐ 100 ☐ Fit to Quantity

☐ Hide Name Columns ☐ PLC Addresses (Base 1)
☐ Address in Cell ☐ Enron/Daniel Mode

Request
 RTU
 ASCII

Modbus Poll - [Mbpoll1]

File Edit Connection Setup Functions Display View Window Help

05 06 15 16 17 22 23 TC ? ?

Tx = 238: Err = 0: ID = 1: F = 04: SR = 1000ms

	Name	00000
0		
1		269
2		483
3		
4		
5		
6		
7		
8		
9		

Communication Traffic

Exit Stop Clear Save Copy L

Tx:000495-01 04 00 01 00 02 20 0B
 Rx:000496-01 04 04 01 0D 01 E4 6A 60
 Tx:000497-01 04 00 01 00 02 20 0B
 Rx:000498-01 04 04 01 0C 01 E4 3B A0
 Tx:000499-01 04 00 01 00 02 20 0B
 Rx:000500-01 04 04 01 0D 01 E4 6A 60
 Tx:000501-01 04 00 01 00 02 20 0B
 Rx:000502-01 04 04 01 0C 01 E4 3B A0
 Tx:000503-01 04 00 01 00 02 20 0B
 Rx:000504-01 04 04 01 0D 01 E3 2B A2
 Tx:000505-01 04 00 01 00 02 20 0B
 Rx:000506-01 04 04 01 0D 01 E3 2B A2
 Tx:000507-01 04 00 01 00 02 20 0B
 Rx:000508-01 04 04 01 0C 01 E3 7A 62
 Tx:000509-01 04 00 01 00 02 20 0B
 Rx:000510-01 04 04 01 0D 01 E3 2B A2

5. Address Setting

Function = 06, Address = 257 (0x0101), Value = 1(New Address = 1)

The screenshot displays the Modbus Poll software interface. The main window shows a status bar at the top with the text "Tx = 347: Err = 0: ID = 1: F = 04: SR = 1000ms" and a red "No connection" message. Below this is a table of communication traffic with columns for Name, Address, and Data. The table is currently empty. A "Write Single Register" dialog box is open in the foreground, showing the following settings:

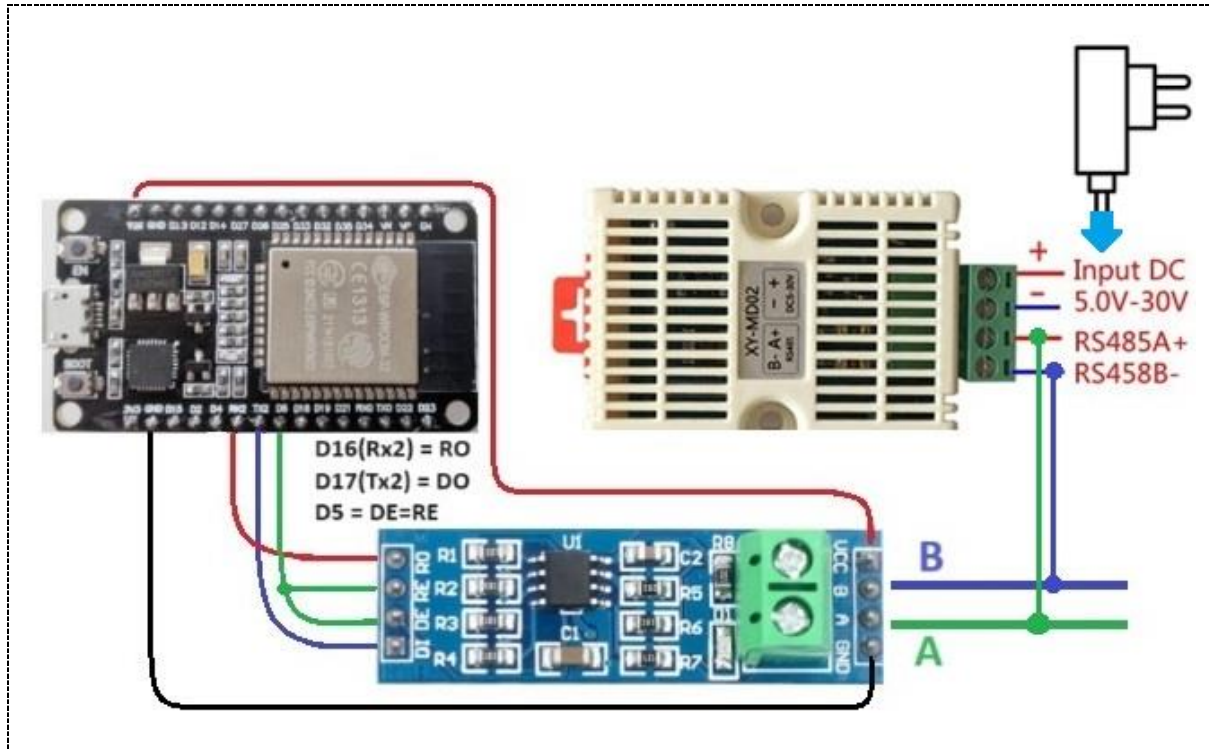
- Slave ID: 11
- Address: 257
- Value: 1
- Result: N/A
- Use Function: ☒ 06: Write single register, ☐ 16: Write multiple registers
- Request: RTU (0B 06 01 01 00 01 18 9C), ASCII (3A 30 42 30 36 30 31 30 31 30 30 31 45 43 0D 0A)

The dialog box also includes "Send" and "Cancel" buttons, and a checkbox for "Close dialog on 'Response ok'".

Appendix 2b/3: Modbus RTU Read

2/6. Read XY-MD02 from DOIT-ESP32-DEVKIT V2

6. Wiring



7. Test Code V1.0 – Read Real Data

```
#define RS485Transmit    HIGH
#define RS485Receive    LOW
#define RS485Control    5 //RS485 Direction control
#define Pin_LEDMonitor  2

byte byteSend;
int StepConut = 0;

void setup() {
  pinMode(RS485Control, OUTPUT);
  pinMode(Pin_LEDMonitor, OUTPUT);
  Serial.begin(9600);
  Serial2.begin(9600);
  digitalWrite(RS485Control, RS485Receive);
  Serial.println("Start Test MODBUS RTU");
}

byte Request[] = {0x01, 0x04, 0x00, 0x01, 0x00, 0x02, 0x20, 0x0B};

void loop() {
  Serial.print("\nTest(");
  Serial.print(++StepConut);
  Serial.print(") > ");
  digitalWrite(Pin_LEDMonitor, HIGH);
  digitalWrite(RS485Control, RS485Transmit);
  delay(10);
  for (int i = 0; i < sizeof(Request); i++)
    Serial2.write(Request[i]);
  delay(10);
  digitalWrite(RS485Control, RS485Receive);
  digitalWrite(Pin_LEDMonitor, LOW);

  for (long int i = 0; i < 40000; i++) {
    if (Serial2.available()) {
      byteSend = Serial2.read();
      if (byteSend < 0x10) Serial.print("0");
      Serial.print(byteSend, HEX);
      Serial.print(" ");
    }
    delayMicroseconds(100);
  }
}
```

8. Test Code V2.0 – Read Real Data and Show

```
#define RS485Transmit    HIGH
#define RS485Receive    LOW
#define RS485Control    5 //RS485 Direction control
#define Pin_LEDMonitor  2

int Wr_Index, StepConut = 0;
byte Request[] = {0x01, 0x04, 0x00, 0x01, 0x00, 0x02, 0x20, 0x0B};
byte Echo[20];

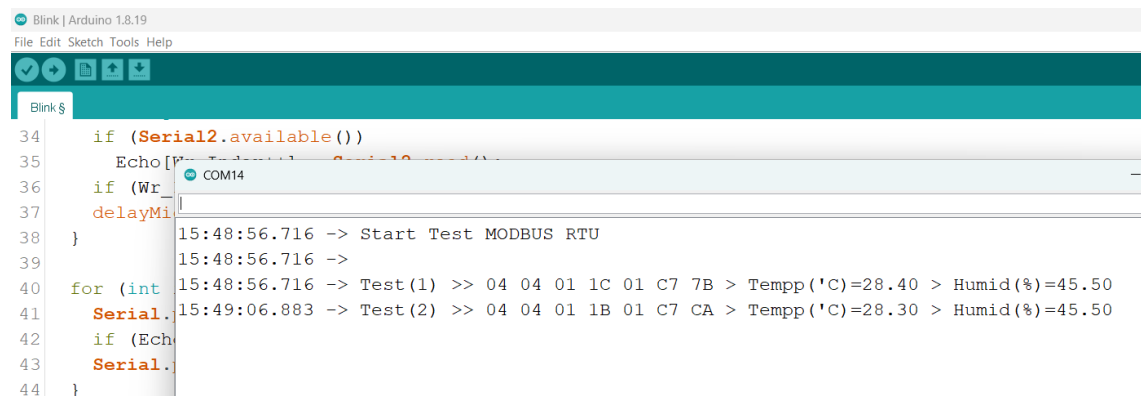
void setup() {
  pinMode(Pin_LEDMonitor, OUTPUT);
  pinMode(RS485Control, OUTPUT);
  Serial.begin(9600);
  Serial2.begin(9600);
  digitalWrite(RS485Control, RS485Receive);
  Serial.println("Start Test MODBUS RTU");
}

void loop() {
  Serial.print("\nTest(");
  Serial.print(++StepConut);
  Serial.print(">>");
  digitalWrite(Pin_LEDMonitor, HIGH);
  digitalWrite(RS485Control, RS485Transmit);
  delay(10);
  for (int i = 0; i < sizeof(Request); i++)
    Serial2.write(Request[i]);
  delay(10);
  digitalWrite(RS485Control, RS485Receive);
  digitalWrite(Pin_LEDMonitor, LOW);
  Wr_Index = -1;
  for (long int i = 0; i < 300000; i++) {
    if (Serial2.available())
      Echo[Wr_Index++] = Serial2.read();
    if (Wr_Index > 12) i = 999999;
    delayMicroseconds(10);
  }

  for (int i = 0; i < Wr_Index - 1; i++) {
    Serial.print(" ");
    if (Echo[i] < 0x10) Serial.print("0");
    Serial.print(Echo[i], HEX);
  }

  float Tempp = (Echo[2] * 256 + Echo[3]) / 10.0;
  Serial.print(" > Tempp('C)="); Serial.print(Tempp, 2);
  float Humid = (Echo[4] * 256 + Echo[5]) / 10.0;
  Serial.print(" > Humid(%)="); Serial.print(Humid, 2);

  delay(5000);
}
```



The screenshot shows the Arduino IDE interface. The top bar indicates 'Blink | Arduino 1.8.19'. The menu bar includes 'File', 'Edit', 'Sketch', 'Tools', and 'Help'. The toolbar shows icons for saving, running, and other functions. The code editor displays the test code from the previous block, with line numbers 34 to 44 visible. A serial monitor window is open, showing the output of the code. The output starts with '15:48:56.716 -> Start Test MODBUS RTU', followed by '15:48:56.716 ->', and then two test results: '15:48:56.716 -> Test(1) >> 04 04 01 1C 01 C7 7B > Tempp('C)=28.40 > Humid(%)=45.50' and '15:49:06.883 -> Test(2) >> 04 04 01 1B 01 C7 CA > Tempp('C)=28.30 > Humid(%)=45.50'. A dropdown menu is visible over the code editor, showing 'COM14' as the selected serial port.

```
34   if (Serial2.available())
35     Echo[Wr_Index++] = Serial2.read();
36   if (Wr_Index > 12) i = 999999;
37   delayMicroseconds(10);
38 }
39
40 for (int i = 0; i < Wr_Index - 1; i++) {
41   Serial.print(" ");
42   if (Echo[i] < 0x10) Serial.print("0");
43   Serial.print(Echo[i], HEX);
44 }
```

Serial Monitor Output:

```
15:48:56.716 -> Start Test MODBUS RTU
15:48:56.716 ->
15:48:56.716 -> Test(1) >> 04 04 01 1C 01 C7 7B > Tempp('C)=28.40 > Humid(%)=45.50
15:49:06.883 -> Test(2) >> 04 04 01 1B 01 C7 CA > Tempp('C)=28.30 > Humid(%)=45.50
```

9. Test Code V3.0 – Read with Library

- Add ModbusMaster by Doc Walker V2.0.1 and test this code

<https://github.com/4-20ma/ModbusMaster>

ModbusMaster
by Doc Walker 4-20ma@wvfans.net> Version 2.0.1 **INSTALLED**
Enlighten your Arduino to be a Modbus master. Enables communication with Modbus slaves over RS232/485 (via RTU protocol).
Requires an RS232/485 transceiver.
[More info](#)
Select version ▾ Install

```
Blink $
25 modbus.pre
26 modbus.pos
27 }
28
29 long lastMil
30 void loop()
31     long curre
32     if (curre
```

COM14
16:04:33.421 -> Temperature: 28.00 C
16:04:33.421 -> Humidity: 47.00 %
16:04:34.415 ->
16:04:34.415 -> Temperature: 28.00 C
16:04:34.415 -> Humidity: 47.00 %

```
#include "ModbusMaster.h" //https://github.com/4-20ma/ModbusMaster

#define Slave_ID 1
#define MAX485_RE_NEG 5
#define RX_PIN 16
#define TX_PIN 17

ModbusMaster modbus;

void preTransmission() {
    digitalWrite(MAX485_RE_NEG, HIGH); //Switch to transmit data
}

void postTransmission() {
    digitalWrite(MAX485_RE_NEG, LOW); //Switch to receive data
}

void setup() {
    pinMode(MAX485_RE_NEG, OUTPUT);
    digitalWrite(MAX485_RE_NEG, LOW);
    Serial.begin(9600, SERIAL_8N1);
    Serial2.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);
    modbus.begin(Slave_ID, Serial2);
    modbus.preTransmission(preTransmission);
    modbus.postTransmission(postTransmission);
}

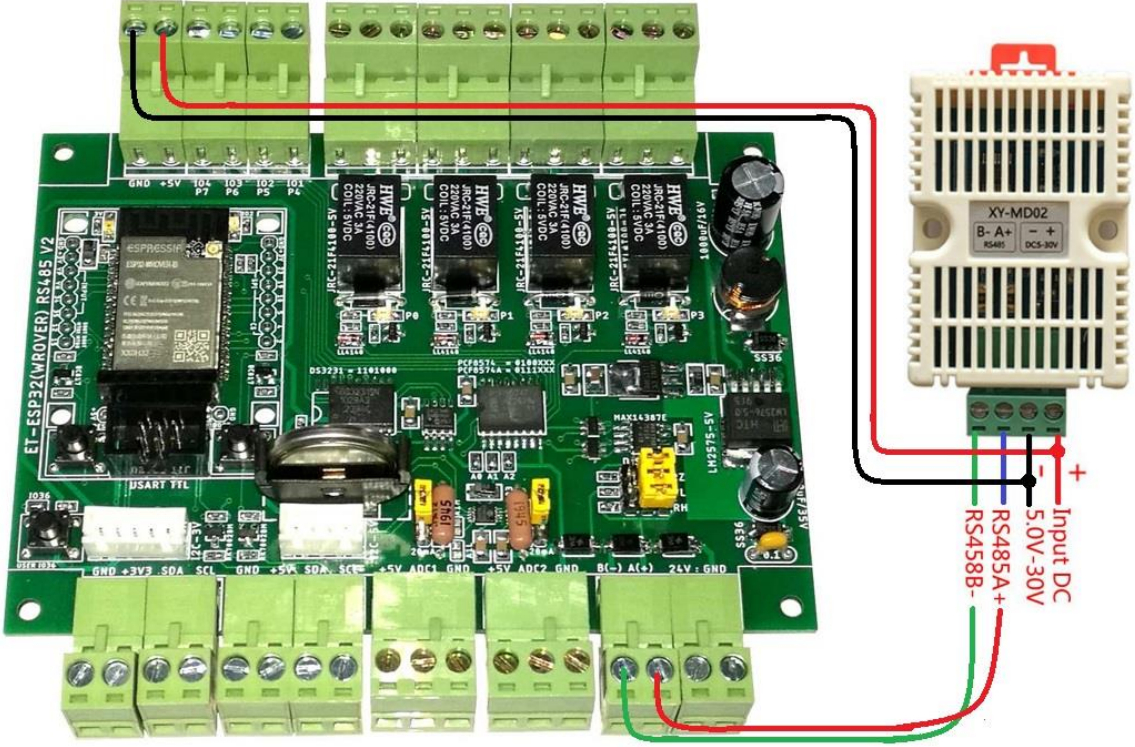
long lastMillis = 0;
void loop() {
    long currentMillis = millis();
    if (currentMillis - lastMillis > 1000) {
        uint8_t result = modbus.readInputRegisters(1, 2);
        if (getResultMsg(&modbus, result)) {
            Serial.println();
            double res_dbl = modbus.getResponseBuffer(0) / 10;
            String res = "Temperature: " + String(res_dbl) + " C\r\n";
            res_dbl = modbus.getResponseBuffer(1) / 10;
            res += "Humidity: " + String(res_dbl) + " %";
            Serial.println(res);
        }
        lastMillis = currentMillis;
    }
}
```

```
bool getResultMsg(ModbusMaster *node, uint8_t result) {
    String tmpstr2 = "\r\n";
    switch (result) {
        case node->ku8MBSuccess:
            return true;
            break;
        case node->ku8MBIllegalFunction:
            tmpstr2 += "Illegal Function";
            break;
        case node->ku8MBIllegalDataAddress:
            tmpstr2 += "Illegal Data Address";
            break;
        case node->ku8MBIllegalDataValue:
            tmpstr2 += "Illegal Data Value";
            break;
        case node->ku8MBSlaveDeviceFailure:
            tmpstr2 += "Slave Device Failure";
            break;
        case node->ku8MBInvalidSlaveID:
            tmpstr2 += "Invalid Slave ID";
            break;
        case node->ku8MBInvalidFunction:
            tmpstr2 += "Invalid Function";
            break;
        case node->ku8MBResponseTimedOut:
            tmpstr2 += "Response Timed Out";
            break;
        case node->ku8MBInvalidCRC:
            tmpstr2 += "Invalid CRC";
            break;
        default:
            tmpstr2 += "Unknown error: " + String(result);
            break;
    }
    Serial.println(tmpstr2);
    return false;
}
```


Appendix 2c/3: Modbus RTU Read

3/6.Read XY-MD02 from ET-ESP32-RS485

10. Test Board



- https://www.etteam.com/product/2C_RS485/ET-ESP32-RS485_V2/index.html
- Select Board = ESP32 WOver Module
-

```
int testLED = 2;

void setup() {
  pinMode(testLED, OUTPUT);
}

void loop() {
  digitalWrite(testLED, HIGH);
  delay(1000);
  digitalWrite(testLED, LOW);
  delay(1000);
}
```

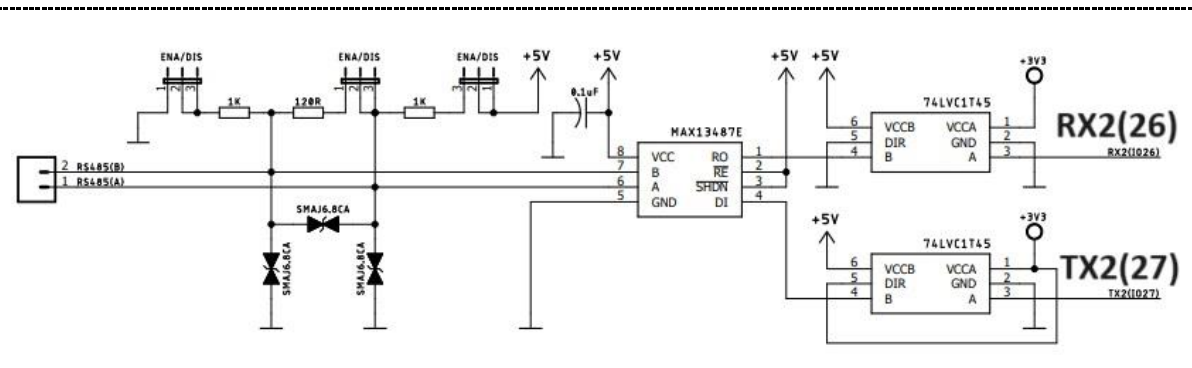
11. Test Code V1.0 – Read with Library

- Add ModbusMaster by Doc Walker V2.0.1 and test this code

<https://github.com/4-20ma/ModbusMaster>

ModbusMaster
by Doc Walker 4-20ma@wvfans.net> Version 2.0.1 **INSTALLED**
Enlighten your Arduino to be a Modbus master. Enables communication with Modbus slaves over RS232/485 (via RTU protocol). Requires an RS232/485 transceiver.
[More info](#)

Select version ▾ Install



```
XXXXXX
25   res_db
26   res +=
27   Serial 16:33:09.628 -> Temperature: 26.00 C
28   }      16:33:09.675 -> Humidity: 49.00 %
29   lastMill 16:33:10.648 ->
30   }        16:33:10.648 -> Temperature: 26.00 C
31   }        16:33:10.648 -> Humidity: 49.00 %
32           16:33:11.628 ->
33           16:33:11.628 -> Temperature: 26.00 C
34 bool getResu 16:33:11.674 -> Humidity: 49.00 %
35 String tmp   16:33:12.650 ->
36 switch (re   16:33:12.650 -> Temperature: 26.00 C
37   case nod   16:33:12.650 -> Humidity: 49.00 %
38   return     16:33:13.630 ->
```

```
#include "ModbusMaster.h" //https://github.com/4-20ma/ModbusMaster
```

```
#define Slave_ID 1
#define RX_PIN 26
#define TX_PIN 27
```

```
ModbusMaster modbus;
```

```
void setup() {
  Serial.begin(9600, SERIAL_8N1);
  Serial2.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);
  modbus.begin(Slave_ID, Serial2);
}
```

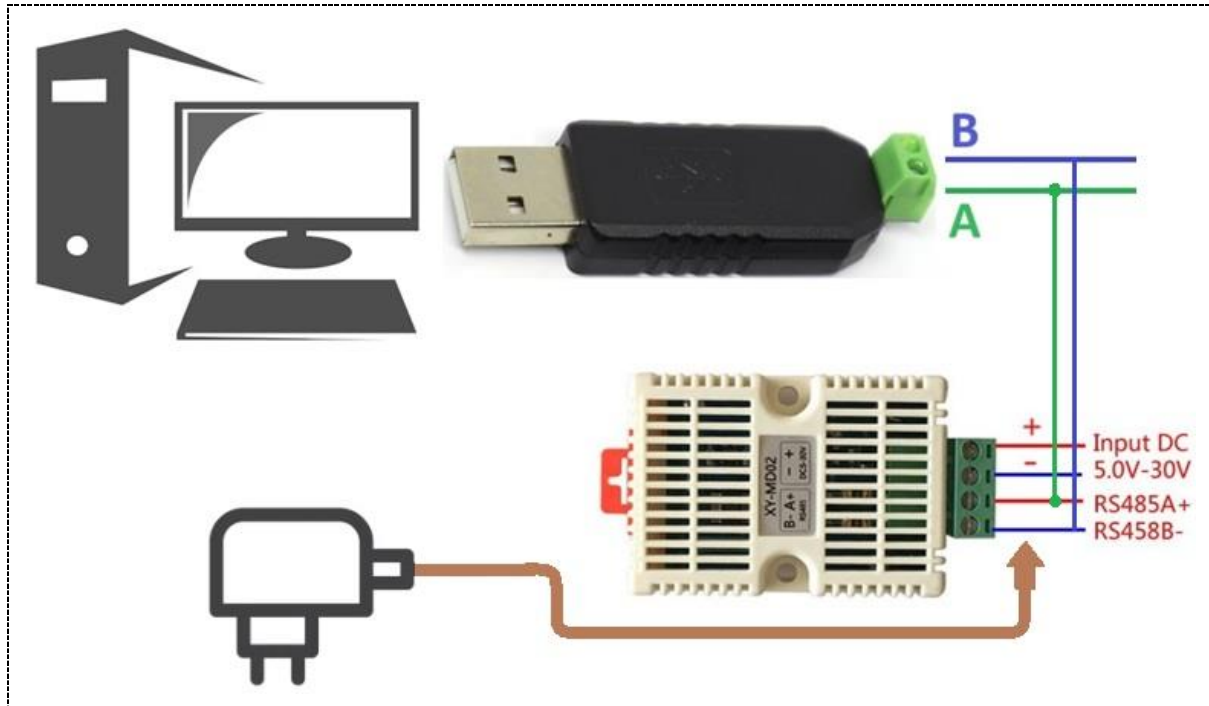
```
long lastMillis = 0;
void loop() {
  long currentMillis = millis();
  if (currentMillis - lastMillis > 1000) {
    uint8_t result = modbus.readInputRegisters(1, 2);
    if (getResultMsg(&modbus, result)) {
      Serial.println();
      double res_dbl = modbus.getResponseBuffer(0) / 10;
      String res = "Temperature: " + String(res_dbl) + " C\r\n";
      res_dbl = modbus.getResponseBuffer(1) / 10;
      res += "Humidity: " + String(res_dbl) + " %";
      Serial.println(res);
    }
    lastMillis = currentMillis;
  }
}
```

```
bool getResultMsg(ModbusMaster *node, uint8_t result) {
  String tmpstr2 = "\r\n";
  switch (result) {
    case node->ku8MBSuccess:
      return true;
      break;
    case node->ku8MBIllegalFunction:
      tmpstr2 += "Illegal Function";
      break;
    case node->ku8MBIllegalDataAddress:
      tmpstr2 += "Illegal Data Address";
      break;
    case node->ku8MBIllegalDataValue:
      tmpstr2 += "Illegal Data Value";
      break;
    case node->ku8MBSlaveDeviceFailure:
      tmpstr2 += "Slave Device Failure";
      break;
    case node->ku8MBInvalidSlaveID:
      tmpstr2 += "Invalid Slave ID";
      break;
    case node->ku8MBInvalidFunction:
      tmpstr2 += "Invalid Function";
      break;
    case node->ku8MBResponseTimedOut:
      tmpstr2 += "Response Timed Out";
      break;
    case node->ku8MBInvalidCRC:
      tmpstr2 += "Invalid CRC";
      break;
    default:
      tmpstr2 += "Unknown error: " + String(result);
      break;
  }
  Serial.println(tmpstr2);
  return false;
}
```

Appendix 2d/3: Modbus RTU Read

4/6.Read XY-MD02 from Node-RED

12. This Circuit



13. Install Node-Red >> <https://levelup.gitconnected.com/installing-node-red-locally-on-a-windows-computer-7570a893633>

14. Download >> <https://nodejs.org/en/>

15. Check our npm and Node.js versions using the Windows Command Prompt.

16. installed Node.js and npm, we can install Node-RED.

```
node --version && npm --version
```

```
npm install -g --unsafe-perm node-red
```

```
npm install node-red
Microsoft Windows [Version 10.0.22631.3296]
(c) Microsoft Corporation. All rights reserved.

C:\Users\peak_>node --version && npm --version
v20.11.1
10.2.4

C:\Users\peak_>npm install -g --unsafe-perm node-red
[ ] \ idealTree:npm: timing idealTree:#root Completed in 2488ms
```

17. Start Node-RED using the command:

node-red

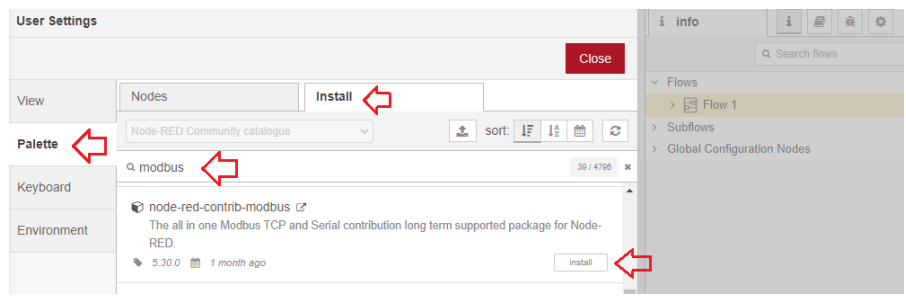
```
C:\Users\peak_>node-red
19 Mar 18:38:09 - [info]

Welcome to Node-RED
=====

19 Mar 18:38:09 - [info] Node-RED version: v3.1.7
19 Mar 18:38:09 - [info] Node.js version: v20.11.1
19 Mar 18:38:09 - [info] Windows NT 10.0.22621.1 x64 UTF-8
```

18. Server Start at <http://127.0.0.1:1880/>

19. Add Palette



The screenshot shows the 'User Settings' dialog in Node-RED. The 'Nodes' tab is selected, and the 'modbus' search filter is applied. The 'node-red-contrib-modbus' package is listed, and its 'Install' button is highlighted with a red arrow. The 'modbus' search filter is also highlighted with a red arrow.

Manage Palette

Palette

Install

Modbus filter

Install

20. Import Code

```
[{"id":"2fb4e09a443b70b6","type":"tab","label":"Flow 1","disabled":false,"info":"","env":[]},{id:"523499d4ad029d1e","type":"modbus-read","z":"2fb4e09a443b70b6","name":"","topic":"","showStatusActivities":false,"logIOActivities":false,"showErrors":false,"showWarnings":true,"unitid":"1","dataType":"InputRegister","adr":"1","quantity":"2","rate":"","rateUnit":"","delayOnStart":true,"startDelayTime":"","server":"4d364e0f1aa3bcf9","useIOFile":false,"ioFile":"","useIOForPayload":false,"emptyMsgOnFail":false,"x":330,"y":220,"wires":[["fccbc6b633d875c0"],[]]},{id:"fccbc6b633d875c0","type":"debug","z":"2fb4e09a443b70b6","name":"debug 1","active":true,"tosidebar":true,"console":false,"tostatus":false,"complete":"false","statusVal":"","statusType":"auto","x":620,"y":120,"wires":[]},{id:"4d364e0f1aa3bcf9","type":"modbus-client","name":"","clienttype":"simpler","bufferCommands":true,"stateLogEnabled":false,"queueLogEnabled":false,"failureLogEnabled":true,"tcpHost":"127.0.0.1","tcpPort":"502","tcpType":"DEFAULT","serialPort":"COM3","serialType":"RTU","serialBaudrate":"9600","serialDatabits":"8","serialStopbits":"1","serialParity":"none","serialConnectionDelay":"100","serialAsciiResponseStartDelimiter":"0x3A","unit_id":"1","commandDelay":"1","clientTimeout":"1000","reconnectOnTimeout":true,"reconnectTimeout":"2000","parallelUnitIdsAllowed":true,"showErrors":false,"showWarnings":true,"showLogs":true}]
```

21. Double Click “Modbus Read” Edit Serial Port

Edit Modbus-Read node

Delete

Cancel

Done

⚙ Properties

⚙

📄

🖨

Settings

Optionals

Name

Topic

Unit-Id

FC

Address

Quantity

Poll Rate

Delay to activate input

Delay Time

Server

Name

Topic

1

FC 4: Read Input Registers

1

2

1-65535

☒

10

modbus-serial@COM3:9600

⌵

🔧

22. Deploy

Flow 1

+

⌵

Modbus Read

active ()

debug 1

⌵

Deploy

⌵

debug

⌵

all nodes

all

msg : string[19]

"modbusinject.failed"

3/19/2024, 6:49:19 PM node: 523499d4ad029d1e

msg : string[19]

"modbusinject.failed"

3/19/2024, 6:49:45 PM node: debug 1

polling : msg.payload : array[2]

▶ [286, 503]

3/19/2024, 6:49:46 PM node: debug 1

polling : msg.payload : array[2]

▶ [286, 502]

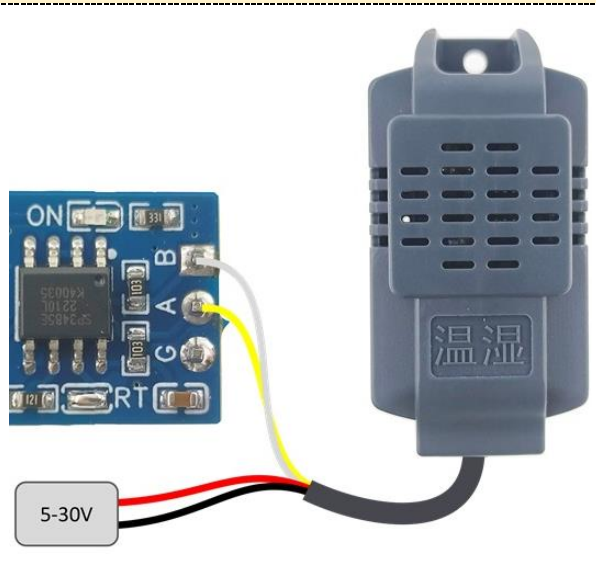
3/19/2024, 6:49:47 PM node: debug 1

polling : msg.payload : array[2]

▶ [286, 502]

Appendix 2e/3: Modbus RTU Read

5/6.Read SHT20



ข้อมูลการเชื่อมต่อ RS485

- Baud rate: 9600
- Data length: 8 bits
- Parity: None
- Stop bit: 1
- Modbus Device ID: 1 (ค่าเริ่มต้น)

MODBUS protocol:

Functional codes used in the product:

0x03: Read hold register

0x04: Read the input register

0x06: Write a single hold register

0x10: Write multiple hold registers

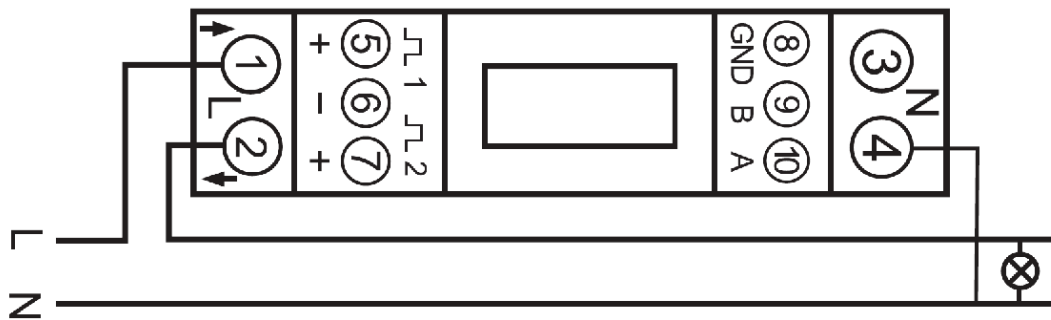
Register type	Register address	Data content	Number of bytes
Input register	0x0001	Temperature value	2
	0x0002	Temperature value	2
Hold register	0x0101	Device address (1-247)	2
	0x0102	Baud rate 0:9600 1:14400 2:19200	2
	0x0103	Temperature correction (/10)10.0~10.0	2
	0x0104	Humidity correction (/10)10.0~10.0	2

1. Read

- <https://www.cybertice.com/product/4802/sht20-เซ็นเซอร์วัดอุณหภูมิและความชื้น-temperature-and-humidity-transmitter-sht20-rs485-sensor-module>
- <https://www.artronshop.co.th/product/687/เซ็นเซอร์วัดอุณหภูมิและความชื้น-modbus-rs485-sht20>

Appendix 2f/3: Modbus RTU Read

6/6.Read SDM120



1. Read

- https://iot-kmutnb.github.io/blogs/sensors/power_meter_sdm120_modbus/

Appendix 3/3: Arduino Cloud Platform



1. Read

- <https://docs.arduino.cc/arduino-cloud/>
- <https://www.ee-diary.com/2022/10/beginner-guide-to-arduino-iot-cloud.html>
-

2. Start